

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
KOMENSKÉHO 44, 040 01 KOŠICE

DEDUSHKA - 2D HRA V UNITY

BENJAMÍN DEÁK

2025

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
KOMENSKÉHO 44, 040 01 KOŠICE

DEDUSHKA - 2D HRA V UNITY

Záverečná práca

Praktická časť odbornej zložky maturitnej skúšky

Forma: Obhajoba vlastného projektu

2025
Košice

Riešitelia:
Benjamín DEÁK, IV.E

Ročník štúdia: štvrtý
Konzultant:
Ing. Ľudmila ĎURATNÁ

ŠPECIFIKÁCIA PROJEKTU NA PRAKTICKÚ ČASŤ ODBORNEJ ZLOŽKY MATURITNEJ SKÚŠKY

Forma : Obhajoba vlastného projektu

Šk. rok: 2024/2025

RIEŠITEĽ

Meno a priezvisko:

Benjamín Deák

Trieda: IV.E

Študijný odbor a zameranie:

2561 M informačné a sieťové technológie - vývoj aplikácií

Hlavný konzultant:

Ing. Ľudmila Ďuratná

Názov projektu:

Dedushka - 2D hra v Unity

Charakteristika projektu:

Cieľom projektu je 2D hra vytvorená pomocou Unity založená na prežívaní vln nepriateľov. Grafika hry bude v štýle pixelartu s pohľadom z hora. Hlavná postava bude slovanský dedo v stredovekom prostredí. Dedo má vlastnú chatrč, no akonáhle ju opustí je obkľúčený nepriateľmi.

Úlohy:

- Navrhnuť a vytvoriť grafické prvky hry
- Navrhnuť a naprogramovať pohyb a animácie postáv
- Navrhnuť a naprogramovať systém boja, na blízko a na diaľku
- Navrhnuť a vytvoriť rôzne typy nepriateľov a typov zbraní
- Navrhnuť a zrealizovať systém farmárčenia pre doplnenie života hráča
- Navrhnuť rôzne typy vylepšení a naprogramovať systém ich získavania
- Vyrobiť zvukové efekty
- Naprogramovať úvodné menu
- Napísať dokumentáciu vo Worde a vytvoriť prezentáciu v PowerPointe

Schválil:

Hlavný konzultant: *Ľ. Ďuratná*

Dátum schválenia: 18. 10. 2024

Vedúci PK: *Ľ. Ďuratná*

Dátum schválenia: 18. 10. 2024

Prevzal (podpis a dátum):

Riešiteľ projektu: *Benjamín Deák*

Spoluriešiteľ projektu:

Obsah

0	Úvod.....	4
1	Problematika a prehľad literatúry	5
1.1	Prehľad existujúcich riešení	6
1.2	Použité technológie.....	8
2	Ciele práce.....	10
2.1	Čiastkové ciele práce	10
2.2	Kritériá kvality.....	11
3	Materiál a metodika.....	12
3.1	Postup práce.....	12
3.2	Spôsob testovania	13
4	Výsledky práce.....	15
4.1	Popis hry	15
4.2	Pohyb	15
4.3	Bojový systém	16
4.4	Mechaniky	16
4.5	Grafika	17
4.6	Menu	17
4.7	Audio	18
5	Diskusia.....	19
6	Závery práce.....	21
7	Zhrnutie	22
8	Zoznam použitej literatúry	23
9	Prílohy.....	I
	Príloha A – Menu	I
	Príloha B – Hra.....	III
	Príloha C – Kódy.....	IV

0 Úvod

Od malička máme silný vzťah k hrám. Milujeme ich a dokážeme sa pri nich uvoľniť. Ďalšou našou záľubou je stredovek a stredoveká tematika. Vždy nám to prišlo veľmi fascinujúce. Hry so stredovekou tematikou patria k našim najobľúbenejším a nedokážeme sa ich nabažiť.

Na strednej škole nás upútalo programovanie, najmä programovanie aplikácií. Vnímali sme to ako možnosť naučiť sa niečo nové a ako možnosť programovania toho čo nás baví. Tak sme si vytvorenie hry stanovili ako takú našu výzvu. Za cieľ sme si teda dali vytvoriť plnohodnotnú 2D hru, ktorej prostredie, postavy a mechaniky sa odohrávajú v stredoveku.

V našej hre sa hráč ocitne ako dedo v jeho chatrči. Má na výber zo štyroch zbraní – sekera, meč, luk, kuša, pričom všetky majú rôzne vlastnosti. Na jeho pozemku sa taktiež nachádza postava druida, vďaka ktorej hráč získava vylepšenia za body. Taktiež je tu aj farma, vďaka ktorej si hráč dokáže doplniť stratené životy. Akonáhle hráč opustí priestory pozemku spúšťa sa vlna nepriateľov a hráč bojuje o život. Za zabíjanie nepriateľov získava body, ktoré si môže uplatniť u druida. Farma sa mu po každej vlne obnoví. Ak hráč vyjde znova a spusti novú vlnu, nepriateľov už bude viac. Úlohou hráča je prebojovať sa k čo najväčšiemu počtu prežitých vln.

Hru chceme nahrat' na Itch.io, čo je otvorený trh indie hier. Veríme, že vzbudí záujem a bude úspešná. V budúcnosti sa plánujeme k hre vrátiť a vylepšiť ju, aby záujem o hru stále rástol a tak aj naše skúsenosti s vývojom.

1 Problematika a prehľad literatúry

Vytvorenie samostatnej a plne funkčnej hry tohto žánru zahŕňa viaceré kľúčové oblasti, ako je výber vývojového prostredia, návrh a implementácia herného ovládania, správanie nepriateľov, bojový systém, systém životov, vylepšenia, mechaniky vln, a tvorba grafických prvkov.

Pri výbere vývojového prostredia existujú rôzne možnosti. Jednou z nich je Unreal Engine, ktorý je silným nástrojom, no jeho náročnosť môže byť problémom pre menšie skupiny alebo začiatočníkov. Tento engine je najmä optimalizovaný pre 3D hry s realistickou grafikou, čo nemusí byť ideálne pre všetky typy hier. Ďalšou možnosťou je Godot, ktorý je open-source, priateľský pre začiatočníkov a získava stále väčšiu popularitu. Godot podporuje vývoj 2D aj 3D hier a využíva GDScript, skriptovací jazyk podobný Pythonu. Nakoniec je tu Unity, ktoré je jedným z najpopulárnejších vývojových prostredí. Je známe pre svoju jednoduchosť, rozsiahlu komunitu a množstvo dostupných návodov. Unity podporuje 2D aj 3D hry a používa objektovo orientovaný skriptovací jazyk C#.

Pohyb postavy môže byť realizovaný pomocou vektorov pre horizontálny a vertikálny pohyb a cez preddefinované vstupy ako WASD, šípky alebo joystick. Tieto vstupy sú kombinované s Rigidbody2D a Collider2D pre fyziku. Iné možnosti zahŕňajú pohyb založený na animáciách, interpolácii medzi bodmi, alebo waypointoch, a pohyb riadený kinematikou, kde je potrebná úplná kontrola nad postavou bez fyziky.

V Unity je možné využiť nástroje ako NavMesh, ktoré umožňujú orientáciu AI v prostredí. Existujú aj ďalšie možnosti, ako je AI plánovanie trasy, kde nepriatelia hľadajú najlepšiu cestu k hráčovi, alebo reaktívne správanie pomocou behavior trees, ktoré umožňuje nepriateľom reagovať na rôzne podmienky. Rovnako môže byť správanie nepriateľov riadené fyzikálnymi zákonmi, čo pridáva realistickejší pohyb v prostredí.

Bojový systém môže fungovať na báze takzvaného Trigger (Collider, ktorý nefunguje ako fyzická prekážka, ale ako zaznamenanie dotyku/vstupu). Alternatívne prístupy môžu zahŕňať použitie hitboxov a hurtboxov na presné určenie, ktoré časti postavy môžu zasiahnuť iné objekty. Ďalšou možnosťou je implementovanie špeciálnych schopností, ako sú magické útoky alebo kombinácie útokov, ktoré poskytujú väčšiu dynamiku boja.

Systém životov funguje na funkciách, pri ktorých sa buď život odoberá zásahom alebo život pribúda. Možnosti regenerácie života môže byť manuálne, kde hráč musí vykonať určitú akciu (napríklad vypit' lektvar), ale aj periodické dopĺňanie životov, kde sa regenerujú v určitých časových intervaloch, alebo dopĺňanie životov na základe špecifických podmienok, napríklad po porazení nepriateľa alebo dokončení úrovne. Tento systém môže byť tiež dynamický, kde sa životy obnovujú v závislosti od udalostí v hre.

Mechaniky vln môžu byť realizované rôznymi spôsobmi. Jednou z možností je systém založený na počte živých nepriateľov. Iná možnosť zahŕňa náhodné alebo preddefinované rozmiestnenie nepriateľov v každej vlne. Pokročilejší prístup môže dynamicky prispôbovať vlny na základe výkonu hráča, teda zvyšovať alebo znižovať počet nepriateľov v závislosti od jeho úspešnosti.

Vylepšenia postavy sú zvyčajne realizované prostredníctvom bodového systému, kde hráči získavajú body a investujú ich do rôznych vlastností postavy. Ďalšie metódy zahŕňajú vylepšenia získané prostredníctvom zberu predmetov v hernom svete alebo na základe skúsenostných bodov, ktoré hráči získavajú počas hry.

Spracovanie grafických prvkov môže byť riešené rôzne. Program špecializujúci sa na pixel art je Aseprite. Ak by vývojár chcel iný grafický dizajn než pixel art, môže využiť Gimp, Adobe Photoshop alebo pre prípad 3D hry programy ako Blender.

1.1 Prehľad existujúcich riešení

1.1.1 Vampire Survivors

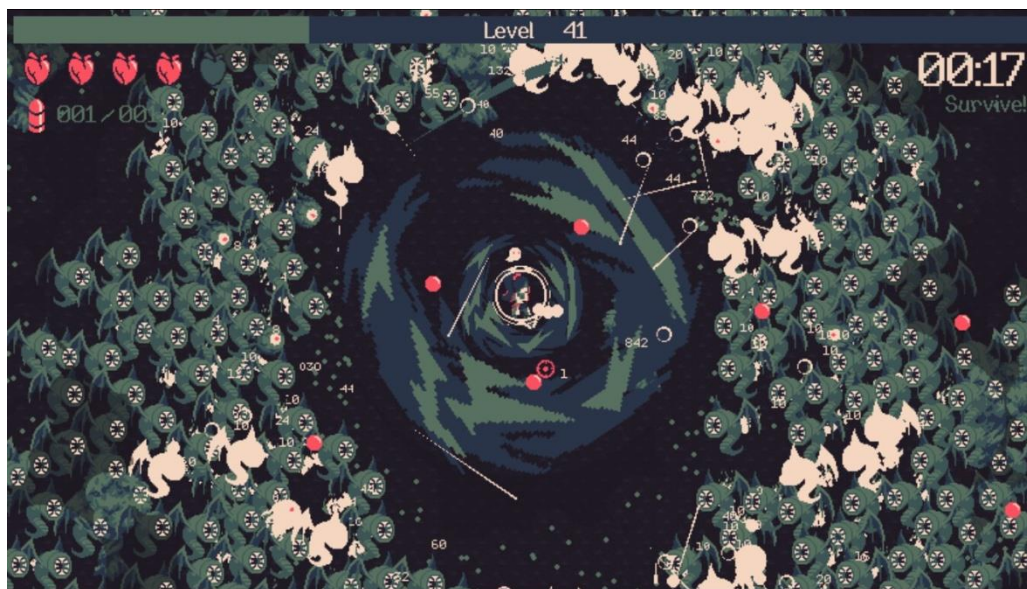
Roguelike shoot 'em up videohra, v ktorej hráč ovláda automaticky útočiacu postavu a snaží sa prežiť čo najviac vln nepriateľov. Postupom cez hru sa hráč snaží odomknúť viacero charakterov, zbraní a relikvií pre použitie v budúcich pokusoch. Každý charakter má iné zbrane a bonusy. Hráč sa môže vylepšiť po každom kole alebo prostredníctvom bední, ktoré získa zabitím špecifických nepriateľov.



Obrázok 1 Vampire Survivors

1.1.2 20 Minutes Till Dawn

Roguelike hra inšpirovaná Vampire Survivors. Založená na prežívaní vln monštier. Počas každého kola hráč získava drahokamy, za ktoré si kupuje vylepšenia, charakterov, zbrane a runy. Hlavný rozdiel od Vampire Survivors je to, že hráč neútočí automaticky.



Obrázok 2 20 Minutes Till Dawn

1.1.3 Porovnanie existujúcich riešení s našou hrou

Hlavné odlišnosti našej hry od spomenutej konkurencie sú rozdielne regenerovanie životov (v našom prípade farma, ktorú musíme manuálne aktivovať po každom kole), jednoduchosť a intuitívnosť bojového systému (jednoduchý sek alebo výstrel, žiadne

šialené kombinácie útokov a bonusov) a rozdielna mechanika spúšťania vln a možnosti v čase medzi vlnami (pozemok deda, jeho opustenie a rôzne interakcie na pozemku).

Výhody nášho projektu:

- Priehľadná jednoduchá grafika (hráčom môže prísť nevoľno pri hraní konkurencie s príliš veľa efektmi a ťažším orientovaním sa v hre).
- Možnosť výmeny zbraní bez penalizácií (zbrane nie sú zafixované na charakter, ale sú jednoducho vymeniteľné počas času medzi vlnami).
- Jednoduchosť herných mechaník (intuitívne ovládanie hry, žiadne obtiažné mechaniky pre fungovanie vln, vylepšenia alebo regeneráciu života).

Pod nevýhody patrí:

- Úzka variácia nepriateľov (máme štyri druhy nepriateľov, čo oproti opozícii je malé číslo), zbraní (štyri druhy zbraní) a vylepšení (tri možné vylepšenia).
- Pomalé tempo hry (tým, že hra nemá rôzne efekty a možnosti rýchleho vyzabíjania, tak tempo hry sa spomaľuje).
- Repetitívnosť (mení sa len počet nepriateľov, zbrane a vylepšenia hráča).

1.2 Použité technológie

Pre programovanie a spojazdnenie samotnej hry sme použili vývojové prostredie Unity. Grafickú stránku hry sme urobili v programe Aseprite, ktorá nám umožnila vytvoriť samostatné assety hry a Photoshop, ktorý nám umožnil doladiť maličkosti.

1.2.1 Unity

Z pestrých možností výberu herného engine-u sme si vybrali Unity. Najviac priateľské vývojové prostredie hier pre začiatočníkov, či už kvôli rozsiahlej komunite, dostupnosti návodov alebo možnosti upravovania projektov v grafickom prostredí. Unity má taktiež možnosť na upravovanie projektu pomocou skriptou napísaných v programovacom jazyku C#, ale aj zabudované rôzne nástroje pre tvorenie fyziky, animácií, UI a prostredia. Ďalšou výhodou je možnosť exportovať hru na rôzne platformy.

1.2.2 Programovací jazyk C#

Samotný Unity engine je napísaný v programovacom jazyku C++, ale pre skriptovanie hier sa využíva jazyk C#. C# je objektovo orientovaný programovací jazyk s ktorým dokážeme jednoducho pracovať aj vďaka grafickému rozhraniu Unity. Pomocou grafického rozhrania Unity sme realizovali väčšinu projektu, napríklad ako: samostatné ovládanie hráča, logiku AI nepriateľov, systém boja, správne fungovanie UI a mechaniky vylepšení, regenerácie, vln.

1.2.3 Aseprite

Grafický program špecializovaný pre techniku pixel art. Vytvorili sme v ňom postavy, chôdzu, útoky a vytvorili ich animácie. Vytvorili sme taktiež celkové prostredie hry, od objektov ako steny, projektily, posteľ, skrinky až po celkovú grafiku hernej tilemapy.

1.2.4 Adobe Photoshop

Photoshop je komerčný grafický program. Primárne je určený na tvorbu a úpravu rastrovej grafiky. Umožňuje tiež použitie grafických efektov a to aj vo forme zásuvných modulov. My sme však tento program využili na prvotné návrhy, úpravu maličkostí a výpomoc s chybami projektu po grafickej stránke.

2 Ciele práce

Naším hlavným cieľom je vytvoriť plnohodnotnú 2D videohru, ktorá hráča upúta a poskytne plynulý herný zážitok. Hra je založená na bojovom systéme s postupnými vlnami nepriateľov, pričom dôraz sa kladie na plynulé ovládanie, strategické využívanie zbraní a postupne narastajúcu obtiažnosť. Výsledný produkt by mal byť technicky stabilný, vizuálne priehľadný a dostatočne vyvážený, aby hráča motivoval k napredovaniu hrou a získavaniu vylepšení.

2.1 Čiastkové ciele práce

Na naplnenie hlavného cieľa sme si stanovili tieto čiastkové ciele:

- Vytvorenie priehľadnej a jednoduchej grafiky technikou pixel art. Návrh a nakreslenie postavy hráča a nepriateľov, vytvorenie objektov prostredia (steny, posteľ, skrinky, ...), navrhnutie a implementovanie tzv. tiles pre prostredie hry.
- Zrealizovať jednotlivé časti animácií útokov a pohybu hráča a AI nepriateľov, následne ich pospájanie do celku, implementovanie pomocou animатора do prostredia hry.
- Doladenie a spojenie grafiky, pohybu a bojového systému nepriateľov. Rôzny výber zbraní pre hráča s rôznymi hodnotami poškodenia. Viacero druhov nepriateľov s inými zbraňami a vlastnosťami.
- Navrhnuť a implementovať systém boja na blízko, ale aj na diaľku. Taktiež spojzdenie vln nepriateľov.
- Sfunkčnenie systému života pre hráča a pre nepriateľov. Vytvorenie funkcií pre uberanie života a doplnenie života. Prepojenie grafiky a mechaník farmy s funkciou doplnenia života hráča.
- Vymyslenie rôznych druhov vylepšenia. Ich implementovanie a prepojenie so svetom, ošetrovanie ich získania.

- Vytvorenie úvodného menu. Spracovanie dizajnu, základnej funkcionality a nadmerných možností.
- Zostavenie grafického rozhrania a jeho funkcionality.
- Implementovanie soundtrack-u a zvukových efektov.

2.2 Kritériá kvality

Na vytvorenie hry sme si stanovili tieto kritéria:

- Plynulé a bezproblémové bežanie hry.
- Správne fungovanie mechaník hry.
- Grafika má byť priehľadná, plynulá a funkčná.
- Nepriatelia majú mať funkčné správanie a reagovať vhodne.
- Menu má byť ľahko ovládateľné a priehľadné.

3 Materiál a metodika

Naštudovali sme si potrebné materiály v Unity manuáli a pomohli sme si s prekonávaním prekážok pomocou návodov na Youtube.

3.1 Postup práce

1. Prvotné navrhovanie a plánovanie

Na začiatku sme si sadli a generovali rôzne nápady a prvotné vízie hry pomocou Brainstormingu. Zmýšľali sme sa nad žánrom hry. Vytvorili sme prvé návrhy grafiky, skúšali sme rôzne štýly (pohľad spredu, 3/4 pohľad, kreslená grafika). Potom sme vymýšľali mechaniky a spôsoby fungovania bojového systému, doplnenia života, vylepšení.

2. Grafické prvky

Ďalším krokom bolo nakresliť vzhľad postáv a vytvorenie animácií pre chôdzu a útoky. Následne sme sa pustili do dizajnovania tiles hracej plochy. Na základe vzhľadu hracej plochy sme sa rozhodli nakresliť objekty (napr.: postel'), UI a menu. Pripravili sme si všetky potrebné grafické prvky, aby sme sa k tomu nemuseli vrátiť a mohli pokračovať plynulo vo vývoji hry.

3. Základné mechaniky

Zkonštruovali sme prvotnú mechaniku chodenia hráča a nakoniec sme ju aj zdokonalili. K tomu sme pridali aj otáčanie sa hráča za kurzorom myši. Ďalej sme spojzdnili pohyb AI nepriateľov a ich otáčanie. Chôdzu sme prepojili s animáciami v animatore. Vytvorili sme systém boja na základe trigger-a, kde sme implementovali útočenie hráča a útočenie umelej inteligencie. Všetko to sme taktiež prepojili s animáciami a vytvorili sme ďalšie druhy zbraní a ich rozdielnú funkčnosť.

4. Hracia plocha

Následne sme sa pustili do zostavenia hracej plochy pomocou tiles. Vytvorili sme širokú mapu a dosadili do nej objekty, pridelili sme im Collider2D a taktiež sme im doladili vlastnosti. Zdefinovali sme neprechodné okraje mapy a vytvorili sme možnosť interakcií s objektmi.

5. Úvodné menu

Po grafickej stránke sme rozmiestnili tlačidlá a modelovali vzhľad úvodného menu. Vytvorili sme viacero prechodov (úvodné menu, nastavenia, ovládanie). K tlačidlám sme nakódovali funkcie a prepojili ich. Úvodné menu sme následne prepojili s hlavnou scénou hry.

6. Herné rozhranie

Po úvodnom menu sme sa rozhodli pre vytvorenie ďalších menu. Ako prvé sme sa pustili do pause menu, pokračovalo menu výberu zbraní a nakoniec menu vylepšení. Menu výberu zbraní sme prepojili so správnou zmenou charakteru a jeho ovládania. Menu vylepšení sme prepojili s vlastnosťami hráča. Obe tieto menu sme taktiež prepojili s interakciami objektu. Následne po dokončení menu sme sa pustili do UI, kde sme vytvorili funkčné grafické rozhranie pre život a počet bodov/nepriateľov.

7. Pokročilé mechaniky

Prepojili sme interakcie objektov s hráčom a naprogramovali sme systém vln na báze porovnania počtu nepriateľov a polohy hráča. K tomu sme pridali body získané za každé eliminovanie AI nepriateľa, ktorými sme ošetrili získavanie vylepšení.

8. Audio

Implementovali sme soundtrack do úvodného menu a do pozadia hry. Taktiež sme implementovali a nastavili zvukové efekty, aby spĺňali naše požiadavky.

9. Doladenie chýb a testovanie

Hru sme hraním testovali a každú nájdenú chybu opravili, či už išlo o nesprávne fungovanie mechaník, grafické vady alebo chyba v spojení.

3.2 Spôsob testovania

1. Testovanie mechaník jednotlivo

Každá mechanika je testovaná pri jej vývoji, aby sa zabezpečilo jednotlivé fungovanie mechaník. Aby nevznikali zbytočné problémy a aby nastalo správne fungovanie aj medzi mechanikami.

2. Testovanie mechaník spoločne

Testovanie mechaník spoločne, aby sa zamedzilo nesprávnemu chodu hry. Hranie a skúšanie rôznych vecí, ktoré by mohli viesť k nesprávnemu fungovaniu hry. Prebieha počas vývoja hry, aby sa hra postupne fungovala správne.

3. Hranie

Hranie a skúšanie hry hráčmi, ktorí nemajú na vývoji podiel. Väčšia šanca, že skupina hráčov nájde chybu počas hrania a oboznámia vývojára. Vývojár vďaka nájdeným nedostatkom skupinou môže viac sústrediť svoju prácu na opravu chýb.

4 Výsledky práce

4.1 Popis hry

V našej hre hráč predstavuje postavu reprezentujúcu slovanského deda. Ten má vlastnú chatrč a pozemok obkolesený hradbami. V chatrči má stojan na zbrane, z ktorého si dokáže vybrať štyri druhy zbraní – sekeru, meč, luk a kušu. Každá zbraň má rôzne atribúty, ako rýchlosť a jej silu. Sekera je najpomalšia, ale najsilnejšia. Kuša sa nabíja dlhšie ako luk, ale projektil letí rýchlejšie a nepriateľovi uškodí viac. Meč je rýchlejší, ale slabší variant sečnej zbrane, taktiež má aj štít, ktorý vie zastaviť projektil. Najslabší ale rýchlejší ako kuša je luk.

Na pozemku sa nachádza aj farma, vďaka ktorej si hráč dokáže doplniť stratené životy zo súboja. Táto farma sa obnovuje po každej zdolanej vlne nepriateľov. Farma je grafický objekt s ktorým vieme interagovať. Ak je farma pripravená na použitie graficky uvidíme rastliny na roli, ak nie je, uvidíme len riadky hlíny.

Okrem farmy sa na pozemku nachádza taktiež postava druida, alebo v slovanskej tematike – postava žreca. Interakcia s druidom privedie hráča do menu, kde si za získané body môže kúpiť vylepšenia. Vie si vylepšiť svoju silu, rýchlosť a počet životov.

Ak hráč opustí pozemok a vyjde za hradby spustí vlnu nepriateľov. Každá nasledujúca vlna sa stupňuje v obtiažnosti, teda počet nepriateľov pribúda. Nepriatelia prichádzajú v štyroch druhoch - so sekerou, s mečom, lukom alebo kušou. Zabíjanie nepriateľov a prežitie vlny sú hlavné mechaniky hry. Hráč za zabíjanie získava body, ktoré si môže uplatniť u druida. Po skončení vlny sa hráč môže vrátiť na pozemok, vyliečiť sa, zmeniť si zbraň alebo interagovať s druidom. Opätovné opustenie pozemku vedie k nasledovnej obtiažnejšej vlne nepriateľov. Toto je hlavný cyklus hry, kde sa hráč ma prebojovať čo k najvyššiemu počtu prežitých vln. Obrázky vzhľadu hry sa nachádzajú v prílohe B.

4.2 Pohyb

Tak ako celá hra, tak aj pohyb hráča je naprogramovaný v jazyku C#. Pohyb hráča je založený na získavaní hodnôt cez horizontálny vector a vertikálny vector. Herný engine Unity má predvolené získavanie týchto hodnôt pomocou tlačidla WASD, šípky alebo joystick páčky. Tieto hodnoty následne normalizujeme pre správne fungovanie, respektíve aby nevzniklo to, že sa hráč bude pohybovať rýchlejšie diagonálne. AI pohyb

funguje na základe polohy hráča. AI získa polohu hráča a následne sa k nemu priblíži (v prípade AI s útokom na diaľku sa môže aj oddialiť). Získavanie polohy je v krátkych periodických intervaloch. Využívame Rigidbody 2D a Collider 2D pre implementáciu fyziky.

```
void Update()
{
    direction = new Vector2(Input.GetAxisRaw("Horizontal"), Input.GetAxisRaw("Vertical")).normalized;
    AnimateMovement(direction);

    mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
}

Unity Message | 0 references
private void FixedUpdate()
{
    rb.MoveRotation(Quaternion.LookRotation(Vector3.forward, mousePos - transform.position));
    rb.MovePosition(rb.position + direction * speed * Time.fixedDeltaTime);
}
```

Obrázok 3 Pohyb hráča

4.3 Bojový systém

Bojový systém je rozdelený na dva typy – na blízko a na diaľku. Na blízko funguje na báze takzvaného Trigger (Collider, ktorý nefunguje ako fyzická prekážka, ale ako zaznamenanie dotyku/vstupu). Ak hráč sa priblíži k nepriateľovi so sečnou zbraňou príliš blízko (spustí trigger), tak AI vykoná útok, ktorý následne ak zasiahne hráča (hráč sa stále nachádza v perifériách trigger-a), tak mu uberie nastavenú hodnotu životov. Podobne to funguje aj zo strany hráča, ktorý však útok spúšťa kliknutím myšou a nie automaticky vzdialenosťou nepriateľa. Útok na diaľku funguje natiiahnutím (nabitím) zbrane a vystrelením. Pri vystrelení sa zjaví objekt (šíp alebo šípka), ktorý ak trafi (spustí trigger) daný cieľ (hráča alebo nepriateľa), tak mu uberie životy a objekt projektilu zmizne. Pri AI sa zbraň prebija vždy, keď je prázdna a výstrel závisí od vzdialenosti hráča (ak je príliš ďaleko, tak sa priblíži a až tak vystrelí). Hráč strieľa a prebija manuálne klikom myši.

4.4 Mechaniky

Interakcie sú založené tiež na báze trigger-a, kde ak sa hráč priblíži k danému objektu s ktorým vie interagovať, tak sa odomkne možnosť stlačenia tlačidla pre interakciu a stlačenie následne spúšťa interakciu (či už doplnenie života alebo otvorenie menu). Tak isto funguje aj objavovanie strechy alebo zablokovanie vstupu po východe z pozemku, kde hráč opustí daný trigger.

Samostatné spúšťanie vln a končenie funguje na základe kontrolovania živých nepriateľov a polohy hráča, teda ak hráč je mimo svoj pozemok a nie je žiadne AI živé, tak sa objavia nepriatelia. Navyšovanie obtiažnosti vln funguje na náhodnej inkrementácii (v rozsahu 0-2 každého druhu) počtu nepriateľov.

Vylepšenia pozostávajú z troch druhov – sila, počet životov a rýchlosť pohybu. Sú ošetrené tým, že na ich získanie potrebujeme body, ktorých hranica po každom vylepšení vstupne o 10. Bod získame za každé zabitie nepriateľa. Body sú po aktivovaní vylepšenia odrátané o daný počet bodov, koľko bol potrebný pre kúpu vylepšenia.

```
void LogicSpawn(NpcScript npc, ref int npcCount)
{
    npcCount += Random.Range(0, 3);

    for(int i = 0; i < npcCount; i++)
    {
        npc.SpawnNpc();

        NpcScript newNpc = npc.GetComponent<NpcScript>();
        npcs.Add(newNpc);
    }
}
```

Obrázok 4 Spawn NPC

4.5 Grafika

Grafika je jednoduchá a priehľadná, robená formou pixel art. Všetko je prispôsobené na Top-down kameru. Je ľahko rozlíšiť hlavnú postavu od nepriateľov. Máme rôzne druhy nepriateľov, ktorí sa líšia tým, akú majú zbraň. Animácie sú robené tak, aby bolo jasne vidieť akú aktivitu charakter vykonáva, či už pohyb alebo útok. Je možné jednoducho odlíšiť rôzne druhy objektov prostredia (postel', steny, skrinky) od zeme prostredia. Farebne je to prispôsobené, aby to nevadilo očiam a aby sa postavy v prostredí nestrácali. UI je robené čo najviac jednoducho, funkčne a priehľadne.

4.6 Menu

Išlo nám o jednoduchosť, funkčnosť a priehľadnosť viacerých druhov menu. Napríklad úvodné menu, ktoré obsahuje nastavenia, ovládanie, pokračovanie a opustenie hry alebo pause menu, ktoré funguje na báze zamrzenia hry, aby sa predišlo nepríjemnostiam. Taktiež rôzne menu interakcií ako menu vylepšení alebo menu výberu zbrane. Všetky tlačidlá v menu fungujú správne (v prípade vylepšení sú aj obstarané bodmi), zamrazia

čas, môžeme sa vrátiť do hry, opustiť celkovo hru alebo sa vrátiť do menu. Obrázky príkladov menu sa nachádzajú v prílohe A.

4.7 Audio

Implementovanie soundtrack-u a zvukových efektov, tak aby sa hodili do tématiky hry, či už zvuk sečných zbraní, výstrelu alebo len tlačidiel. Soundtrack je taktiež prispôbený stredovekej tématike. Všetky potrebné nahrávky sme našli na platforme Freesound alebo Youtube.

5 Diskusia

Prvá prekážka bola tvorenie grafiky. Vyskúšali sme rôzne štýly a rôzne polohy kamery. Vyriešili sme to tak, že sme sa rozhodli pre jednoduchosť a zároveň priehľadnosť. Vytvorili sme grafiku skrz pixel art pre top-down kameru. Proces kreslenia bol aj napriek tomu veľmi zdĺhavý a museli sme sa potrápiť nad správnosťou animácií.

Pohyb bol zo začiatku zasekaný a veľmi divný. Miestami bol rýchlejší a miestami pomalší. Jednoducho sme to ošetrili normalizáciou vektorov a použitím Rigidbody 2D aby postavy podliehali fyzike.

Prepojenie animácie s útokom bolo veľmi namáhavé. Najprv sme to skúšali počkaním pomocou Time, ale to nefungovalo. Nakoniec sme našli Animation Event, čo nám umožňuje spustiť funkciu v danej časti animácie, vďaka ktorej sme zvládli spojiť správne fungovanie útočenia.

Keď sme sa dostali k bojovému systému a interakciám, tak sme nevedeli ako ďalej. Po chvíli hľadania sme zistili, že Collider 2D sa dá prepnúť na trigger. Pomocou toho sme dokázali zostrojiť bojový systém a interakcie. Bolo potrebné vytvoriť podmienku pre objekt nachádzajúci sa v danej oblasti.

Ďalší problém prišiel pri začiatku vlny. AI nepriatelia sa objavovali aj na pozemku hráča, kde by nemali. To sme vyriešili pomocou funkcie, kde sme vypočítali a presunuli NPC mimo oblasť daného trigger-a, teda pozemku deda.

```
private Vector3 MoveOutOfSafezone(Vector3 position)
{
    Bounds safezoneBounds = safezone.GetComponent<Collider2D>().bounds;

    Vector3 direction = (position - safezoneBounds.center).normalized;

    float distanceToMove = Mathf.Max(safezoneBounds.extents.magnitude, 5f);

    position += direction * distanceToMove;

    return position;
}
```

Obrázok 5 Funkcia presunu

Následne sme museli vyriešiť správanie NPC, teda urobiť to, aby nepriatelia so zbraňami na diaľku neprenasledovali hráča. Najprv to bolo veľmi zasekané pretože AI sa neustále držalo čo najďalej od hráča. S tým sme sa trochu potrápili, ale dokázali sme to. Teraz ak

sa priblížite k nepriateľovi s lukom alebo kušou chvíľu počká a následne začne utekať, kde už v maximálnej nastavenej diaľke ostane a nebude sa prebíjať ešte ďalej.

6 Závery práce

Naučili sme sa pracovať s vývojovým prostredím Unity a porozumeli jeho základom a fungovaniu. Vylepšili sme svoje vedomosti v objektovo orientovanom programovaní pomocou programovacieho jazyka C#. Naučili sme sa pracovať s programom Aseprite a oprášili svojho umeleckého ducha. Oboznámili sme sa s animovaním a s jeho sfunkčnením.

Požiadavku grafiky sa nám podarilo splniť, s tým, že máme priehľadne vlastnoručne nakreslené postavy a prostredie, animácie, jednoduché menu a UI.

Zrealizovali sme a prepojili sme jednotlivé animácie s pohybom a útokom hráča, ale aj AI nepriateľov. Máme funkčný pohyb hráča získavaný vstupom z klávesnice alebo ovládača. NPC má taktiež funkčný pohyb, kde nepriatelia na blízko nasledujú hráča a nepriatelia na diaľku si držia odstup.

Máme funkčný bojový systém, či už na blízko skrz daní trigger zbrane alebo aj na diaľku skrz collider projektilu. Taktiež máme správne fungovanie vĺn, ktoré je ošetrené či už správnym „spawnom“ nepriateľov alebo kontrolou počtu nepriateľov a polohy hráča.

Hráč a AI nepriatelia majú správne fungujúci systém života, kde pri útoku sa život uberie o stanovenú hodnotu a pri hráčovi sa pomocou manuálnej interakcie s farmou doplní. Farma má taktiež „cooldown“ a je graficky rozlíšiteľné kedy je ju možno použiť (po každom kole).

Pomocou interakcie s druidom si hráč môže zakúpiť vylepšenia tykajúce sa sily, života a rýchlosti. Pre kúpu vylepšení sú potrebné body, ktoré je možné získať zabitím AI nepriateľov.

Úvodné menu je priehľadné a plné funkčné, tak ako aj pause menu. Navyše k tomu máme ešte menu pre výber zbrane a menu pre kúpu vylepšení.

Máme funkčné a priehľadné UI. Soundtrack je prispôsobený stredovekému štýlu, tak ako aj zvukové efekty.

Hru by sme vedeli rozšíriť druhmi zbraní, nepriateľov alebo vylepšení. Taktiež vytvoriť iné mapy s inými nepriateľmi a inými mechanikami. Rozšíriť tento projekt môžeme aj dodaním systému schopností, alebo upresniť obtiažnosť hry.

7 Zhrnutie

Naším cieľom bolo vytvoriť plnohodnotnú 2D videohru, ktorá hráča upúta a poskytne plynulý herný zážitok.

Hra je vytvorená v top-down pixelart štýle. Má stredovekú tematiku a jednoduché ovládanie, nie je logicky náročná. Je robená tak, aby si človek mohol oddýchnuť. Grafický dizajn je priehľadný a jednoduchý. Hra je vytvorená vo forme pixel art-u, odohráva sa formou top-down pohľadu. Menu je jednoduché a nastavenia jasne poukazujú na ovládanie hry.

Pri tvorbe hry sme sa naučili základy programovania v C# a ovládanie vývojového prostredia Unity, tiež sme sa naučili pracovať s programom Aseprite pre kreslenie pomocou pixel art formy. Práca nás bavila a radi sme boli na druhej strane herného priemyslu.

V budúcnosti by sme chceli spestriť počet druhov nepriateľov, zbraní a vylepšení. Vytvoriť nové mapy s rôznymi prostrediami a mechanikami.

8 Zoznam použitej literatúry

[1] UNITY TECHNOLOGIES. 2024. Unity Manual [online]. 2024 [citované dňa 2025-01-14]. Dostupné na internete:

<<https://docs.unity3d.com/6000.0/Documentation/Manual/>>

[2] UNITY TECHNOLOGIES. 2024. Unity Manual: Rigidbody 2D [online]. 2024 [citované dňa 2025-01-14]. Dostupné na internete:

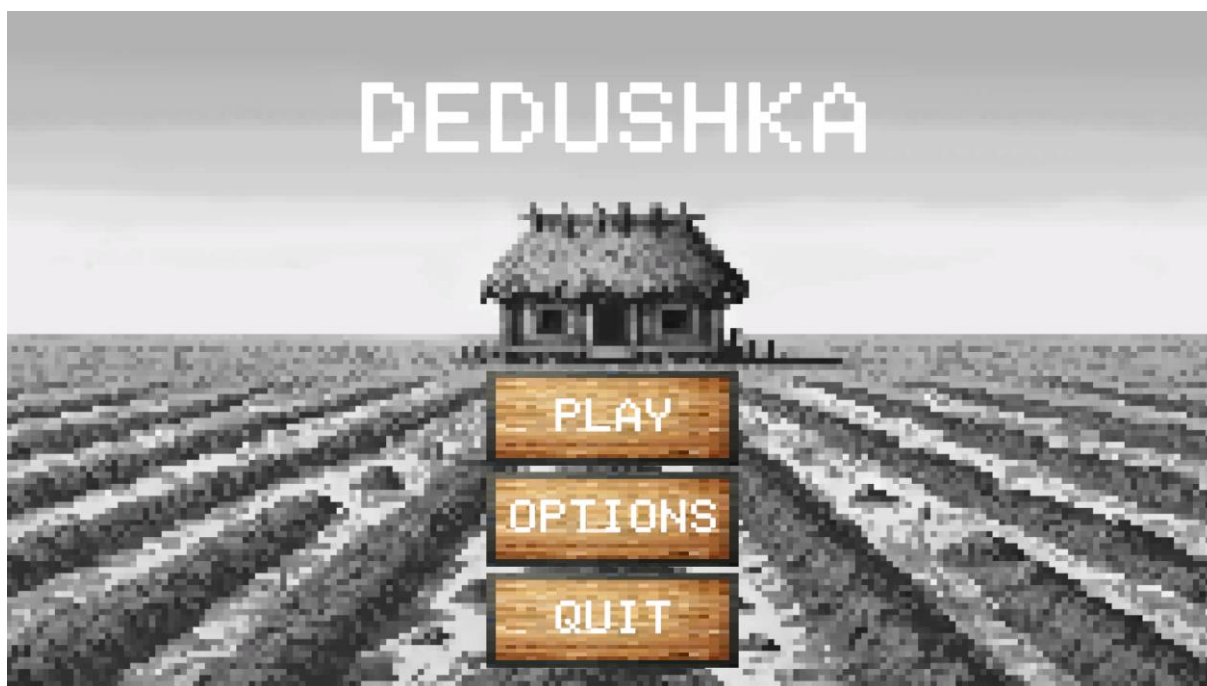
<<https://docs.unity3d.com/6000.0/Documentation/Manual/2d-physics/rigidbody/rigidbody-2d-landing.html>>

[3] UNITY TECHNOLOGIES. 2024. Unity Manual: Collider 2D [online]. 2024 [citované dňa 2025-01-14]. Dostupné na internete:

<<https://docs.unity3d.com/6000.0/Documentation/Manual/2d-physics/collider/collider-2d-landing.html>>

9 Prílohy

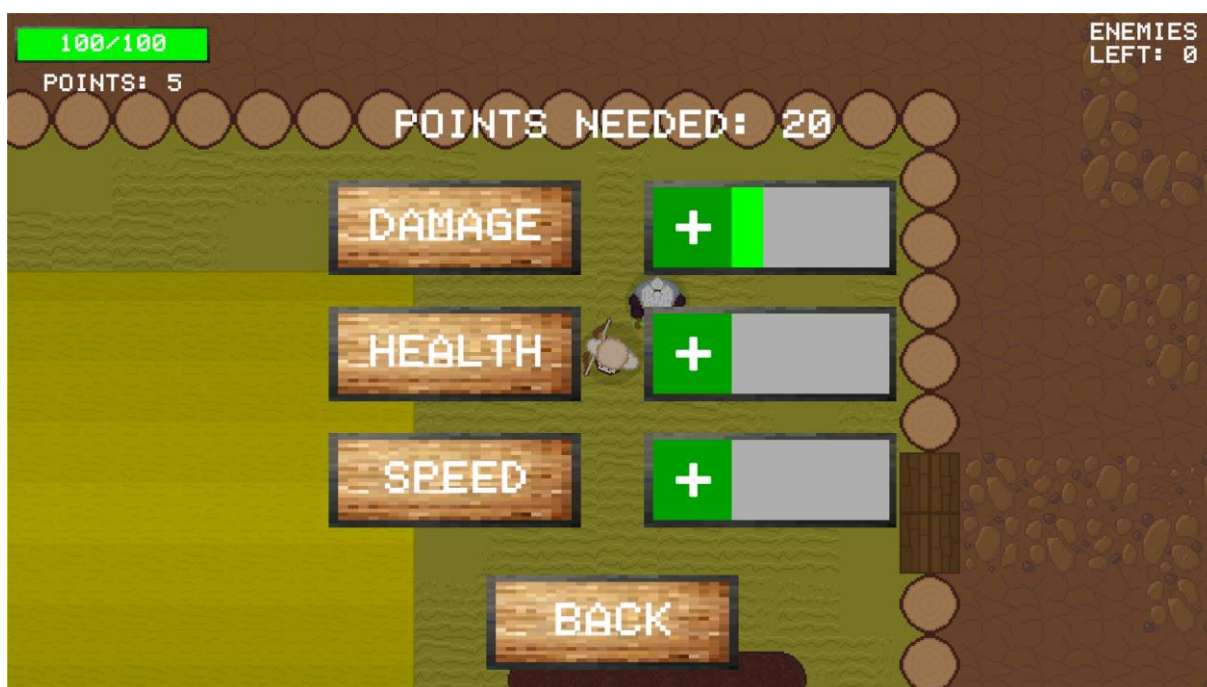
Príloha A – Menu



Obrázok 6 Úvodné menu



Obrázok 7 Pause menu



Obrázok 8 Menu vylepšení

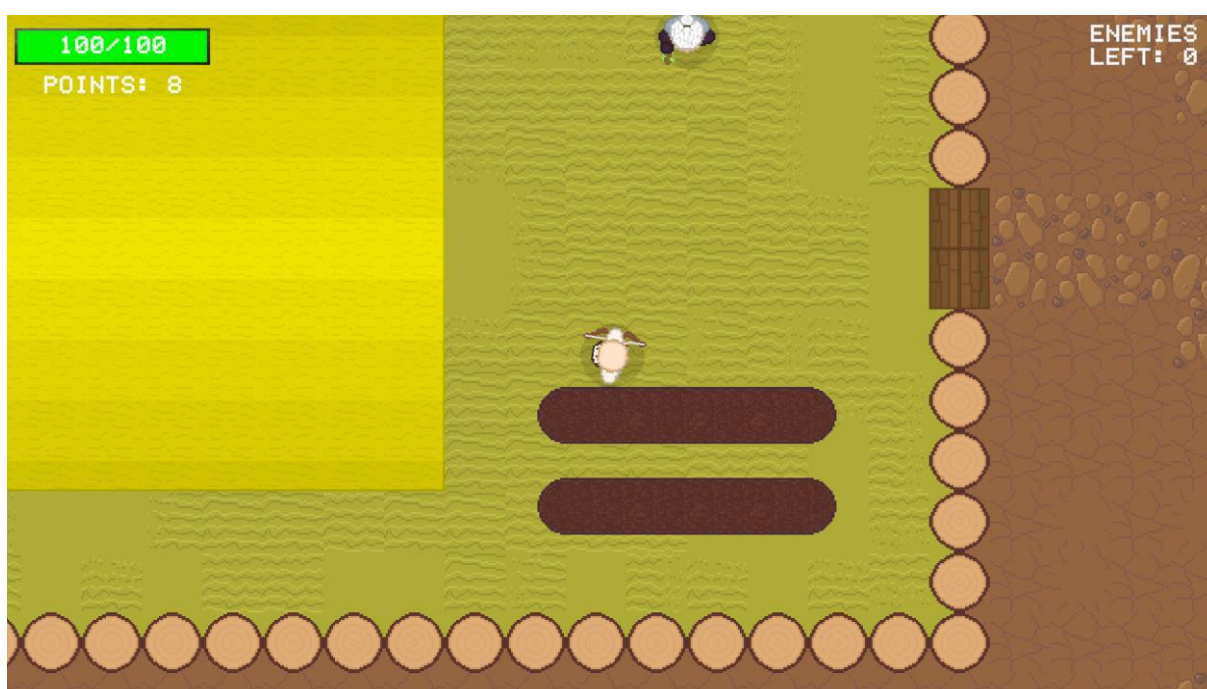


Obrázok 9 Menu výběru zbraní

Príloha B – Hra



Obrázok 10 Boj



Obrázok 11 Safezone (pozemok)

Príloha C – Kódy

```
private void FixedUpdate()
{
    if (target != null)
    {
        rb.linearVelocity = Vector2.zero;
        Vector2 directionToPlayer = (target.position - transform.position).normalized;
        float distanceToPlayer = Vector2.Distance(transform.position, target.position);

        if (npc.CompareTag("npcBow") || npc.CompareTag("npcCrossbow"))
        {
            if (distanceToPlayer > maxFollowDistance)
            {
                rb.MovePosition(rb.position + directionToPlayer * moveSpeed * Time.fixedDeltaTime);
                AnimateMovement(directionToPlayer);
            }
            else if (distanceToPlayer < minFollowDistance)
            {
                stayTimer += Time.fixedDeltaTime;
                if (stayTimer >= stayTimeBeforeRunningAway)
                {
                    rb.MovePosition(rb.position - directionToPlayer * moveSpeed * Time.fixedDeltaTime);
                    AnimateMovement(directionToPlayer);
                }
                else
                {
                    AnimateMovement(Vector3.zero);
                }
            }
            else
            {
                stayTimer = 0f;
                AnimateMovement(Vector3.zero);
            }
        }
        else
        {
            rb.MovePosition(rb.position + directionToPlayer * moveSpeed * Time.fixedDeltaTime);
            AnimateMovement(directionToPlayer);
        }

        RotateTowardsTarget(directionToPlayer);
    }
}
```

Obrázok 12 Pohyb NPC

```

void Update()
{
    meleeReady = true;
    npcAnimator.ResetTrigger("weaponAttack");
    npcAnimator.ResetTrigger("stopWeaponAttack");

    if (Time.timeScale != 0f)
    {
        if (attackTrigger.isPlayerInRange && Time.time >= lastActionTime + actionCooldown)
        {
            Debug.Log("attack");
            lastActionTime = Time.time;

            if (meleeReady == true)
            {
                npcAnimator.SetTrigger("weaponAttack");
                meleeReady = false;
            }

            StartCoroutine(EndAttackAnimation());
        }
    }
}

1 reference
private IEnumerator EndAttackAnimation()
{
    AnimatorStateInfo stateInfo = npcAnimator.GetCurrentAnimatorStateInfo(0);

    while (stateInfo.normalizedTime < 1f)
    {
        stateInfo = npcAnimator.GetCurrentAnimatorStateInfo(0);
        yield return null;
    }

    npcAnimator.SetTrigger("stopWeaponAttack");

    meleeReady = true;
}

```

Obrázok 13 NPC útok na blízko

```

void Update()
{
    if (isInRange)
    {
        if (Input.GetKeyDown(interactKey1))
        {
            interactive1.SetActive(false);
            interactAction1.Invoke();
        }
        if (Input.GetKeyDown(interactKey2))
        {
            interactAction2.Invoke();
        }
    }
}

Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.gameObject.CompareTag("Player"))
    {
        isInRange = true;
        interactive1.SetActive(true);
    }
}

Unity Message | 0 references
private void OnTriggerExit2D(Collider2D collision)
{
    if (collision.gameObject.CompareTag("Player"))
    {
        isInRange = false;
        interactive1.SetActive(false);
        hintInteractive.SetActive(false);
    }
}

```

Obrázok 14 Interakcie